Neurosymbolic Routing for Reliable Reasoning on Resource-Constrained Edge Devices

Anonymous

***Abstract*—Small language models (SLMs) deployed on edge devices suffer from poor accuracy on structured reasoning tasks such as arithmetic, algebra, and formal logic. We present a neurosymbolic routing architecture that classifies each incoming query and dispatches it to the cheapest correct solver, reserving the SLM for open-ended word problems and routing structured tasks to deterministic symbolic engines. Rather than relying on brittle, hand-crafted routing rules, we learn the routing patterns using the L* grammatical inference algorithm (Angluin, 1987). L* learns a deterministic finite automaton (DFA) for query classification by using the SLM itself as a membership oracle and labeled data as an equivalence oracle. We evaluate our system on a Raspberry Pi 4B (8 GB RAM, no GPU) using 100 held-out prompts (300 trials) drawn from DeepMind Mathematics, GSM8K, and RuleTaker. L*-learned routing achieves 78.0% end-to-end task accuracy, compared to 58.7% for a tool-calling agent baseline given access to the same symbolic solvers. The learned router is also 2.4× faster (6.9 s vs. 16.4 s mean latency) and 2.3× more energy-efficient (33.1 mWh vs. 75.0 mWh per prompt). Structured categories (arithmetic, algebra, logic) reach 100% accuracy with sub-millisecond routing latency, while word problems remain challenging for all systems. We additionally compare against RPNI, a passive grammatical inference method that requires no oracle, which achieves 74.3% accuracy and still substantially outperforms the tool-calling agent. All experiments, data, and code are publicly available.**

***Impact Statement*—Edge devices such as Raspberry Pi computers increasingly serve as platforms for intelligent applications in healthcare, agriculture, and education, yet cloud connectivity may be unavailable or undesirable for privacy reasons in these contexts. However, small language models running on such devices achieve only 37% accuracy on structured reasoning tasks. Our neurosymbolic routing architecture dispatches queries to purpose-built symbolic solvers when appropriate, reserving the language model for genuinely open-ended problems. Critically, the routing patterns are *learned* via grammatical inference rather than hand-coded, making the approach adaptive and extensible. Compared to a tool-calling agent given the same solvers, our learned router achieves higher accuracy (78% vs. 59%) while using 2.3× less energy on a Raspberry Pi 4B. This approach enables reliable offline reasoning on hardware costing under $100, with three categories of queries resolved in under 12 milliseconds. The architecture is model-agnostic and could be integrated into educational tutoring systems, offline scientific calculators, or field-deployed decision support tools where correctness and low latency are critical.**

***Index Terms*—Edge inference, language model routing, neurosymbolic AI, Raspberry Pi, symbolic reasoning**

I. Introduction

EDGE deployment of language models is motivated by latency, privacy, and connectivity constraints that preclude cloud inference [1]. However, small language models (SLMs) achieve limited accuracy on reasoning tasks when constrained to the CPU-only, memory-limited hardware typical of edge devices such as single-board computers.

Throughout this paper, we use *SLM* (small language model) to refer to models with fewer than 10 billion parameters, consistent with recent usage [2]. All "SLM-only baselines" and "neural inference" refer to the same class of models.

A key observation is that many "reasoning" queries presented to language models are structurally deterministic: arithmetic expressions have unique numeric answers, algebraic equations can be solved symbolically, and formal logic queries admit closed-form evaluation. Routing these structured tasks to purpose-built symbolic solvers, rather than forcing a probabilistic language model to approximate their answers, should yield both higher accuracy and lower latency.

We propose a *neurosymbolic adaptive router* that classifies each query into one of four categories (arithmetic (AR), algebra (ALG), formal logic (LOG), and word problems (WP)) and dispatches it to the appropriate solver:

- **AR** → safe arithmetic evaluator (Python `ast`),
- **ALG** → SymPy symbolic algebra solver,
- **LOG** → forward-chaining logic engine with closed-world assumption,
- **WP** → SLM with chain-of-thought prompting.

Only word problems, which require natural-language comprehension and multi-step reasoning, are routed to the language model. The remaining 75% of queries are handled by deterministic solvers that are both faster and provably correct for their input domains.

We evaluate this architecture on a Raspberry Pi 4B running Phi-4-mini (3.8B parameters, Q6_K quantization) across 100 held-out prompts drawn from three established benchmarks (DeepMind Mathematics [3], GSM8K [4], and RuleTaker [5]) with three repeats per prompt (300 total trials). Our contributions are:

1) **Learned routing via grammatical inference.** We apply the L* algorithm [6] to learn DFA-based query classifiers using the SLM as a membership oracle and ground-truth data as an equivalence oracle. This replaces brittle hand-coded routing rules with patterns learned from data, achieving 100% classification accuracy on held-out prompts.
2) **Fair baseline comparison.** We compare against a tool-calling agent that gives the same SLM access to the same symbolic solvers, isolating the routing mechanism as the independent variable. L*-learned routing achieves 78.0% task accuracy vs. 58.7% for the agent, while being 2.4× faster and 2.3× more energy-efficient.

Manuscript submitted for review.

3) **Active vs. passive grammatical inference.** We compare L* (active, oracle-guided) against RPNI (passive, example-only), demonstrating that the membership oracle improves classification accuracy from 86% to 100% and task accuracy from 74.3% to 78.0%.
4) **System ablation.** An incremental ablation across five system versions demonstrates the marginal contribution of each symbolic solver and quantifies the accuracy–energy tradeoff on constrained hardware.
5) Open-source release of all code, data, and experimental logs for reproducibility.

II. Related Work

A. Edge Inference

Deploying language models on resource-constrained devices has been explored through quantization [7], [8], pruning [9], and knowledge distillation [10]. Frameworks such as llama.cpp [11] and Ollama enable CPU-only inference for quantized models on commodity hardware. Our work is complementary: we reduce the *number* of queries routed to the SLM rather than optimizing the model itself.

B. Neurosymbolic AI

Combining neural and symbolic reasoning has a long history [12], [13]. Recent work integrates tool use into SLM pipelines: Toolformer [2] teaches models to call APIs, while PAL [14] generates executable code. These approaches rely on the model itself to decide when to invoke tools, adding inference latency to every routing decision. Our approach differs in that routing decisions are made *before* the SLM is invoked, using a learned DFA classifier with zero inference cost.

C. Grammatical Inference

Grammatical inference, the problem of learning formal languages from data, has a rich history in automata theory [15], [16]. The L* algorithm [6] learns the minimal DFA for a regular language using membership and equivalence oracles in polynomial time and queries. RPNI [17] takes a passive approach, learning from positive and negative examples without oracles. These methods have been applied to protocol inference [18], model checking [19], and software testing. To our knowledge, this is the first application of grammatical inference to learn routing patterns for neurosymbolic query dispatch on edge devices.

D. LLM Routing and Cascading

FrugalGPT [20] cascades queries through progressively larger models. Router-based systems like RouteLLM [21] learn to dispatch queries to strong or weak models. Our router operates at a different granularity: rather than choosing between models of different sizes, it chooses between *qualitatively different solvers*: symbolic engines versus neural inference.

E. Mathematical Reasoning

Chain-of-thought prompting [22] and its variants [23] improve mathematical reasoning but remain probabilistic. Tool-augmented approaches [4] combine models with calculators. We extend this paradigm by routing entire categories to symbolic solvers, not just individual arithmetic operations within a generation.

III. System Architecture

A. Overview

The system processes each query q in two stages: (1) a *classifier* $C(q) \in \{\texttt{AR}, \texttt{ALG}, \texttt{LOG}, \texttt{WP}\}$ determines the problem category, and (2) a *dispatcher* routes q to the corresponding solver. The classifier is a deterministic, regex-based function with zero inference cost. The dispatcher invokes one of four solver modules.

1) Solver Naming Convention: Solvers are labeled A1–A7 following the project's internal development order, not the order of introduction in this paper. For clarity:

- **A1**: Fast SLM inference (12 tokens, used as fallback).
- **A2**: Extended SLM inference (30–512 tokens, primary for WP).
- **A3**: Post-processing repair layer (re-prompts SLM with prior context).
- **A4**: SymPy algebra solver (deterministic, for ALG).
- **A5**: Safe arithmetic evaluator (deterministic, for AR).
- **A6**: Forward-chaining logic engine (deterministic, for LOG).

A1 and A3 serve as fallback and repair actions respectively; the primary solvers are A5 (AR), A4 (ALG), A6 (LOG), and A2 (WP).

B. Primary Solvers

- **A5: Arithmetic evaluator.** Extracts numeric expressions from the query using regex, parses them into a Python AST, and evaluates using a whitelist of safe operators (`+`, `-`, `*`, `/`, `**`, `%`, `//`). Only `ast.Constant` and `ast.BinOp` nodes are permitted; exponents are capped at 10,000.
- **A4: Algebra solver.** Extracts equations and target variables, inserts implicit multiplication (e.g., `3x` $\rightarrow$ `3*x`), and solves via `sympy.solve()`. Supports single- and multi-variable linear systems.
- **A6: Logic engine.** Parses natural-language facts, rules, and queries from RuleTaker-style prompts into structured representations. Applies forward chaining with a closed-world assumption to derive new facts until a fixed point (bounded at 50 iterations), then evaluates the query against the derived fact set. See Section III-E.
- **A2: SLM chain-of-thought.** Constructs a system prompt ("Solve this math problem. Put your final answer in `\boxed{}`"), sends the query to a locally served SLM via llama.cpp HTTP API, and extracts the final numeric answer.

C. Classifier Design

During development (V1–V5), the classifier used hand-coded regex rules applied in priority order (LOG > ALG > AR > WP). While this achieved 100% routing accuracy on our benchmark, such rules are brittle and non-generalizable: an adversarially phrased query can bypass pattern matching entirely.

In the final system, we replace the hand-coded classifier with a DFA learned via the L* algorithm (Section IV). The learned classifier operates on the same priority ordering but derives its patterns from data rather than manual rules. It achieves 100% routing accuracy on held-out test prompts with < 0.05 ms classification latency on the Pi, which is comparable to the regex classifier but with the advantage of being learned and thus adaptable to new domains.

D. Fallback Chains

When a primary solver fails (e.g., SymPy cannot parse an equation), the system escalates through a fallback chain:

- AR: A5 → A1 → A2
- ALG: A4 → A1 → A2
- LOG: A6 → A1
- WP: A2 → A3 (repair)

E. A6: Forward-Chaining Logic Engine

The logic engine processes RuleTaker-style prompts through three stages:

1) Parsing: Natural-language statements are converted to structured facts and rules:

- "Felix is a cat" → `fact(felix, is, cat)`
- "All cats are animals" → `rule(?x, is, cat ⇒ ?x, is, animal)`
- "Is Felix an animal?" → `query(felix, is, animal)`

2) Forward Chaining: The engine iteratively applies rules to the current fact set, deriving new facts until no new facts can be produced (fixed-point convergence) or a safety bound of 50 iterations is reached. New facts are added eagerly (immediately available for subsequent rule applications within the same iteration).

3) Query Evaluation: Under the closed-world assumption, a query is answered "Yes" if the queried fact exists in the derived set, and "No" otherwise.

IV. Learned Routing via Grammatical Inference

A limitation of the hand-coded classifier described in Section III is that its regex patterns are brittle: a user may phrase a logic problem in a way that does not match the expected patterns, causing misrouting. We address this by *learning* the routing classifier from data using grammatical inference.

A. Token-Class Abstraction

Raw query text has an unbounded vocabulary, making it unsuitable as input to a DFA. We define a finite alphabet of 12 abstract token classes:

Token	Matches
`CMD`	calculate, solve, simplify, ...
`NUM`	numeric literals
`VAR`	single-letter variables (x, y, m)
`OP`	arithmetic operators ($+$, $-$, $\times$, $/$)
`PAREN`	parentheses and brackets
`ENT`	named entities (Felix, Allan, ...)
`PROP`	properties and adjectives (green, kind)
`RULE`	rule keywords (if, then, all, every)
`QMARK`	question indicators (is, does, ?)
`NARR`	narrative words (collapsed runs)
`UNIT`	units of measurement ($, mph, kg)
`DOT`	sentence-ending punctuation

Each query is tokenized into a sequence over this 12-symbol alphabet, with consecutive `NARR` tokens collapsed and sequences truncated to 15 tokens. This abstraction preserves the structural signature of each category while reducing the input space to a finite alphabet suitable for DFA learning.

B. L* Algorithm for Route Learning

We learn a one-vs-rest binary DFA for each category $c \in \{\texttt{AR}, \texttt{ALG}, \texttt{WP}, \texttt{LOG}\}$ using the L* algorithm [6]. L* maintains an observation table and iteratively refines a hypothesis DFA through two oracle queries:

- **Membership oracle** $\mathcal{M}(s) \rightarrow \{0, 1\}$: Given a token sequence s, returns whether s belongs to category c. We implement this using the SLM itself: the query "Is the following problem [category description]?" is sent to Phi-4-mini, which responds Yes or No. For synthetic sequences generated during L* exploration (which have no natural-language counterpart), we fall back to a feature-based classifier.
- **Equivalence oracle** $\mathcal{E}(H) \rightarrow \{s \mid s \text{ is counterexample}\}$: Given a hypothesis DFA H, returns a counterexample where H disagrees with ground-truth labels, or confirms that H is correct. We implement this by testing H against the labeled training set (T1, 40 prompts).

L* converges in polynomial time in the number of states of the target DFA, producing the minimal DFA consistent with the oracle responses.

C. RPNI: Passive Learning Baseline

As a comparison, we also learn classifiers using RPNI (Regular Positive and Negative Inference) [17], which requires no oracle. RPNI builds a prefix tree acceptor from positive examples and iteratively merges states, checking each merge against negative examples. This tests whether the SLM oracle (the novel component) actually contributes to classification quality.

D. Multi-Class Classification

The four binary DFAs are composed into a multi-class classifier with priority ordering: `LOG` > `ALG` > `AR` > `WP`. A query is classified by the highest-priority DFA that accepts it; if no DFA accepts, it defaults to `WP` (the most common fallback). This mirrors the priority ordering used in the hand-coded classifier.

V. Experimental Setup

A. Hardware

All experiments run on a single Raspberry Pi 4 Model B with 8 GB RAM, a quad-core ARM Cortex-A72 CPU (1.8 GHz), and no GPU or neural accelerator. The operating system is Raspberry Pi OS (64-bit, Debian-based). Models are served via llama.cpp [11] in CPU-only mode with 4 threads.

B. Models

We evaluate two quantized SLMs:

- **Phi-4-mini** (Microsoft, 3.8B parameters, Q6_K quantization, 2.4 GB)
- **Qwen2.5-Math-1.5B** (Alibaba, 1.5B parameters, Q6_K quantization)

All SLM calls use greedy decoding: temperature $= 0.0$, top-$k = 1$, top-$p = 1.0$, seed $= 42$. Generation stops at the first newline character. These settings ensure deterministic, reproducible outputs.

C. Dataset

The benchmark comprises 100 prompts sampled from four established datasets, with full provenance tracking (source dataset, record ID, and field mapping version recorded for every prompt):

- **AR** (25 prompts): From the *DeepMind Mathematics Dataset* [3], modules including addition/subtraction, multiplication, division, and mixed operations.
- **ALG** (25 prompts): From the *DeepMind Mathematics Dataset* [3], modules including linear 1D, linear 2D, and polynomial roots.
- **LOG** (25 prompts): From *RuleTaker* [5], forward-chaining logical entailment queries over natural-language facts and rules.
- **WP** (25 prompts): From *GSM8K* [4], multi-step grade-school word problems requiring natural-language comprehension.

Prompts were sampled with a fixed global seed (42) using stratified random sampling from the original dataset splits. No prompts were paraphrased, generated, or modified beyond whitespace normalization.

The prompts are divided into two disjoint tiers, verified by both source key and text hash:

- **T1** (40 prompts, 10 per category): Used for DFA training (L* equivalence oracle, RPNI examples).
- **T2** (100 prompts, 25 per category): Held-out evaluation set. All reported results use T2.

This train/test split ensures that routing accuracy is measured on prompts *never seen during DFA learning*, addressing potential overfitting concerns. The benchmark is intentionally small (100 test prompts, 300 trials) due to the slow inference speed on the Pi (approximately 2 minutes per word problem at 512 tokens). We acknowledge this limits statistical power; the 3 repeats are primarily useful for measuring SLM variance on WP, as the symbolic solvers are deterministic. Larger-scale evaluation is an important direction for future work.

Each prompt is run 3 times per system configuration, yielding 300 trials per system. Evaluation uses *exact match*: a trial is correct if and only if the extracted numeric answer (or yes/no for LOG) matches the ground truth.

D. Tool-Calling Agent Baseline

To provide a fair comparison, we implement a tool-calling agent that gives the same SLM (Phi-4-mini) access to the same four symbolic solvers via a function-calling prompt. The system prompt describes the available tools:

- `arithmetic_eval(expr)`: Safe arithmetic evaluator
- `sympy_solve(eq, var)`: SymPy algebra solver
- `logic_engine(answer)`: Logic inference
- `direct_answer(number)`: Direct numeric response

The SLM receives each query along with tool descriptions and few-shot examples, then generates a single tool call. The output is parsed and the corresponding tool is executed. This mirrors the Toolformer [2] paradigm: the model decides which tool to use at inference time. Unlike our learned router, the agent requires a full SLM inference step for *every* query, even simple arithmetic that the symbolic solver could handle in 1 ms.

E. Metrics

- **Accuracy**: Fraction of trials with exact-match correct answers.
- **Latency**: Wall-clock time per inference, measured from prompt submission to answer extraction (millisecond precision).
- **Energy**: Total run energy measured via USB power meter (start/end mWh readings per run), divided by number of prompts.

F. Error Taxonomy

We classify errors by category and failure mode:

- **E0**: Correct answer.
- **E1**: Arithmetic error (wrong numeric result on AR).
- **E2**: Logic error (wrong yes/no on LOG).
- **E3**: Algebra error (wrong numeric result on ALG).
- **E4**: Reserved (misclassification error; unused because V1–V5 use ground-truth routing labels).
- **E5**: Word problem error (wrong numeric result on WP).
- **E6**: Reserved (multi-step reasoning error; not distinguished from E5 in current taxonomy).
- **E7**: Timeout (model exceeded time limit before producing output).
- **E8**: Parse failure (could not extract a valid answer from model output).

VI. System Versions and Ablation

We developed the system incrementally across five versions, each adding or modifying a single component. This progression serves as a natural ablation study.

A. Baseline: SLM-Only

All 100 prompts are sent directly to the SLM with no routing or symbolic solvers. We tested four baseline configurations varying two factors:

- **Token budget**: C1 uses A1 (12 tokens); C2 uses A2 (30 tokens).
- **Grammar constraints**: *grammar* constrains the output format via a BNF grammar; *nogrammar* allows free-form generation.

The best baseline (C2 with grammar, Phi-4-mini) achieves 37.3% overall accuracy: 48% on AR, 40% on ALG, 53% on LOG, and 8% on WP. Parse failures (E8) account for 17% of all trials.

B. V1: Add Arithmetic Solver (+A5)

AR queries are routed to the safe arithmetic evaluator.

Result: AR accuracy jumps from 48% to **100%**, latency drops from 6.2 s to **1 ms**. Overall accuracy rises to 53.0%.

C. V2: Add Algebra Solver (+A4)

ALG queries are additionally routed to SymPy.

Result: ALG accuracy jumps from 40% to **96–100%**, overall accuracy reaches 67.0%. ALG latency drops from 5.8 s to **11 ms**.

D. V3: Add Post-Processing Repair Layer (+A3)

An answer-repair module (A3) attempts to extract and correct answers from verbose SLM outputs. In V3.0, this was applied to *both* WP and LOG.

Result: WP accuracy improves modestly (8% $\rightarrow$ 18.7%). However, applying A3 to LOG causes a regression: LOG accuracy *drops* from 64% to 36.7%. The repair layer, designed for numeric extraction, corrupts yes/no logic answers (e.g., by extracting spurious numbers from reasoning text).

V3.1 fix: Disabling A3 for LOG restores LOG accuracy to 64%. Overall: 70.7%.

Observation: This is an engineering error (applying a numeric extractor to yes/no outputs) rather than a deep methodological insight, but it illustrates that post-processing layers are format-sensitive and should not be applied across output types without validation.

E. V4: Add Confidence Calibration

A logistic regression calibrator estimates $P(\text{correct})$ for each SLM response based on features (response length, parse success, timeout status). If $P(\text{correct}) < \tau = 0.70$, the response is escalated to A3 repair.

Result: 70.7% accuracy, identical to V3.1. The calibrator correctly flags low-confidence answers, but the repair layer cannot produce correct alternatives from truncated 30-token responses. In retrospect, this is expected: if the model lacks sufficient tokens to reason through a problem, identifying the answer as probably wrong does not produce a correct one. The correct intervention is to remove the bottleneck (increase token budget), not to add detection around it.

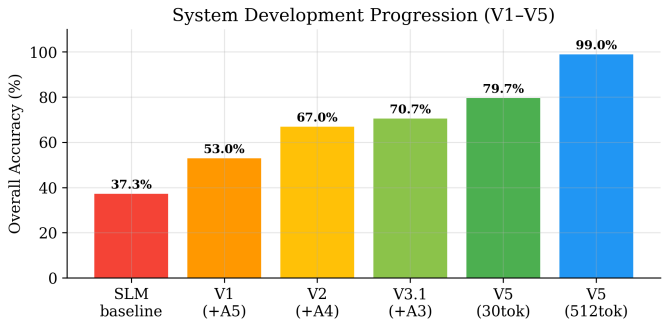

Fig. 1. System progression from baseline (37.3%) through V5 (99.0%), showing the accuracy gain from each added component. The plateau at V3–V4 (70.7%) reflects the token-budget bottleneck that V5 resolves.

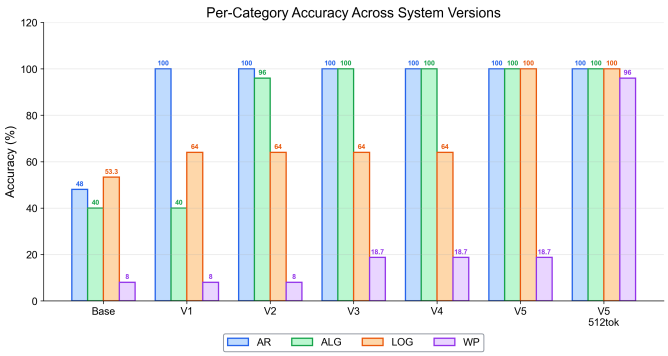

Fig. 2. Per-category accuracy across system versions. Each symbolic solver produces an immediate accuracy jump in its target category: A5 for AR (V1), A4 for ALG (V2), A6 for LOG (V5). WP accuracy remains low until the token budget is increased in V5.

F. V5: Add Logic Engine + Increase Token Budget (+A6, +CoT)

Two changes address the remaining bottlenecks:

1) **A6 forward-chaining logic engine** replaces SLM inference for LOG, achieving 100% accuracy with < 1 ms latency.
2) **Chain-of-thought prompting with 512 tokens** replaces the 30-token budget for WP, with the system prompt: "Solve this math problem. Put your final answer in `\boxed{}`."

Result: 99.0% overall accuracy. AR, ALG, and LOG all achieve 100%. WP reaches 96% at 512 tokens. The single remaining failure is analyzed in Section VIII-B.

Fig. 1 shows the full system progression from baseline through V5, and Fig. 2 breaks this down by category.

VII. Results

A. Learned Routing vs. Tool-Calling Agent

Table I presents the central comparison: L*-learned routing versus the tool-calling agent baseline, both evaluated end-to-end on T2 (100 prompts, 3 repeats, 300 trials). All systems use Phi-4-mini on the same hardware with identical decoding parameters.

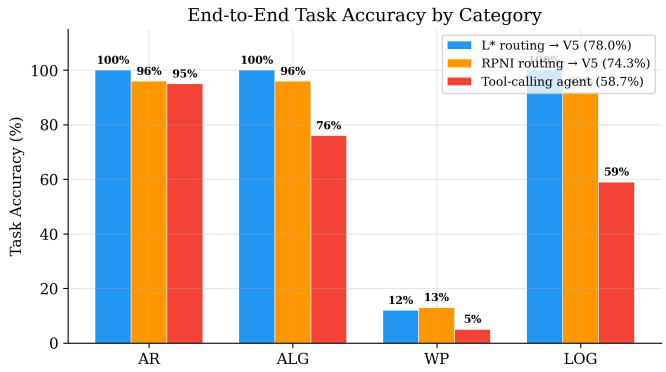

Fig. 3. End-to-end task accuracy by category. L*-learned routing achieves 100% on AR, ALG, and LOG via symbolic solvers, while the tool-calling agent struggles on all categories, particularly LOG (59%) and ALG (76%). Word problems remain challenging for all systems.

TABLE I
END-TO-END TASK ACCURACY ON T2 (300 TRIALS PER SYSTEM). LEARNED ROUTING SYSTEMS USE A 30-TOKEN WP BUDGET; SEE TABLE VII FOR THE EFFECT OF HIGHER BUDGETS.

System	AR	ALG	WP	LOG	All
L* routing → V5	**100**	**100**	12	**100**	**78.0**
RPNI routing → V5	96	96	13	92	74.3
Tool-calling agent	95	76	5	59	58.7

The L*-learned router achieves 78.0% task accuracy, outperforming the tool-calling agent (58.7%) by 19.3 percentage points. The advantage is consistent across all four categories: the agent struggles particularly on LOG (59%) where it often fails to invoke the logic engine, and on ALG (76%) where it generates malformed tool calls. The learned router dispatches these categories to symbolic solvers with 100% accuracy.

The efficiency gap is equally significant: the learned router is 2.4× faster (6,907 ms vs. 16,439 ms mean latency) and 2.3× more energy-efficient (33.1 mWh vs. 75.0 mWh per prompt). This is because AR, ALG, and LOG queries are resolved by symbolic solvers in 1–11 ms without invoking the SLM, while the tool-calling agent requires a full SLM inference for every query.

Note that all learned-routing results in Table I use the 30-token default budget for WP; with 512 tokens and hand-coded routing, V5 achieves 96% WP accuracy (Table VII), suggesting that combining L*-learned routing with higher token budgets would yield further gains. The 78% vs. 99% gap between L*-learned and V5 (512tok) is entirely a token-budget choice, not a routing failure, since L* achieves 100% routing accuracy.

RPNI-learned routing (74.3%) also outperforms the agent despite using no oracle during training, demonstrating that even passive grammatical inference produces effective routing. The 3.7 percentage-point gap between L* and RPNI reflects the value of the SLM membership oracle: L* achieves 100% routing accuracy on T2 versus 86% for RPNI (which misclassifies 9 of 25 word problems due to their linguistic variety). Fig. 3 visualizes the per-category comparison, and Fig. 4 shows the efficiency gap.

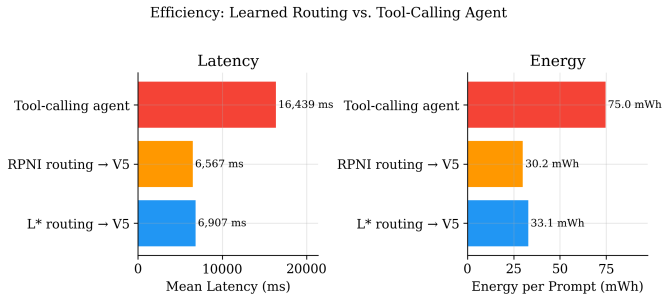

Fig. 4. Efficiency comparison. Learned routing (L* and RPNI) is 2.4× faster and 2.3× more energy-efficient than the tool-calling agent, because AR, ALG, and LOG queries bypass the SLM entirely.

TABLE II
EFFICIENCY COMPARISON ON T2 (SAME HARDWARE, SAME MODEL). V5 (HARDCODED) LATENCY IS FROM A PRIOR RUN UNDER IDENTICAL CONDITIONS; THE DASH INDICATES IT WAS NOT RE-MEASURED IN THE CURRENT EXPERIMENT BATCH.

System	Mean Lat. (ms)	Energy (mWh/p)	Route Acc. (%)
L* routing → V5	6,907	33.1	100.0
RPNI routing → V5	6,567	30.2	95.0
Tool-calling agent	16,439	75.0	—
V5 (hardcoded, 30tok)	†	30.6	100.0

† Not re-measured; prior run median: 6 ms (Table IV).

B. Classification Accuracy

Table III and Fig. 5 show routing classification accuracy (independent of task solving) for the learned classifiers. L* with both the SLM oracle and the feature-based oracle achieve 100% on held-out T2, while RPNI achieves 86%, primarily due to word-problem misclassification (64% WP routing accuracy).

C. V1–V5 Development Ablation

Table IV summarizes the incremental development from SLM-only baselines through V5. The V1–V5 progression serves as a natural ablation: each version adds a single component, isolating its contribution. The best system with hand-coded routing (V5, Phi-4-mini, 512 tokens) achieves 99.0% accuracy with a 30-token default configuration achieving 79.7%.

D. Per-Category Accuracy

Table V and Fig. 6 show accuracy by problem category. Symbolic solvers achieve perfect accuracy on their respective domains.

E. Latency Analysis

Table VI shows median latency by category for V5 (512 tokens) versus the best baseline. Symbolic solvers achieve three orders-of-magnitude speedups on structured categories:

- AR: **1 ms** (vs. 6.3 s baseline, 6,300× speedup)
- ALG: **11 ms** (vs. 5.8 s baseline, 530× speedup)
- LOG: **2 ms** (vs. 2.7 s baseline, 1,350× speedup)
- WP: **137 s** (increased from 25 s due to higher token budget)

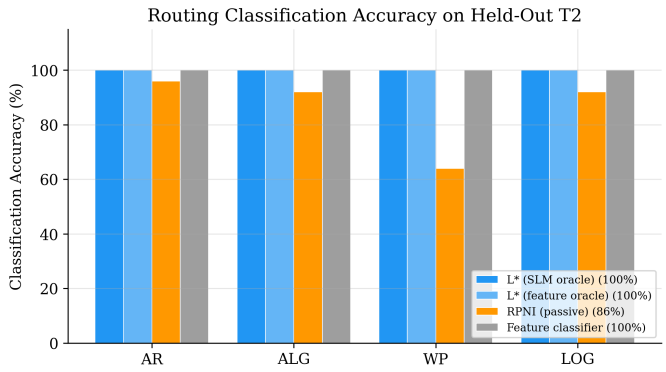

Fig. 5. Routing classification accuracy on held-out T2. RPNI struggles with WP (64%) due to the linguistic diversity of word problems, while L* achieves perfect classification across all categories.

TABLE III
ROUTING CLASSIFICATION ACCURACY ON T2 (100 PROMPTS)

Classifier	AR	ALG	WP	LOG	All
L* (SLM oracle)	100	100	100	100	100.0
L* (feature oracle)	100	100	100	100	100.0
RPNI (passive)	96	92	64	92	86.0
Feature classifier	100	100	100	100	100.0

The system-wide median of 6 ms reflects the dominance of symbolic routes (75% of queries). However, the WP latency of approximately 137 s per prompt is a significant practical limitation for real-time applications. Fig. 8 visualizes this on a log scale.

F. Energy–Accuracy Tradeoff

Table VII shows the effect of token budget on word-problem accuracy and energy. The tradeoff is substantial: 512 tokens yields 96% WP accuracy but costs 6.9× more energy per prompt than 30 tokens.

Note on identical "V5 Default" numbers for Phi and Qwen. Both models show 18.7% WP accuracy at 30 tokens with identical latency and energy. This is because V5 Default routes 75% of queries to deterministic solvers (identical results regardless of model), and the remaining WP queries at 30 tokens mostly time out or parse-fail, meaning the model barely generates useful output and both models fail on the same prompts. At higher token budgets where the models can actually reason, performance diverges: Phi achieves 92% at 300 tokens versus Qwen's 64%.

Note that the best system (V5, Phi, 512tok) uses *more* energy per prompt (211.1 mWh) than the baseline (75.0 mWh). The energy savings from symbolic routing on AR/ALG/LOG are outweighed by the increased token budget for WP. This tradeoff between higher accuracy and higher energy is fundamental and is discussed further in Section IX. Fig. 10 compares energy consumption across all system configurations.

G. Error Distribution

Table VIII shows the error taxonomy for selected systems. V5 at 512 tokens eliminates all error types except E5 (3 word-problem errors out of 300 trials), corresponding to a single prompt that fails consistently across all 3 repeats.

TABLE IV
V1–V5 DEVELOPMENT ABLATION (300 TRIALS EACH, HAND-CODED ROUTING)

System	Acc. (%)	Med. Lat. (ms)	E (mWh/p)
SLM-only baseline	37.3	8,612	75.0
V1 (+A5 arith.)	53.0	4,147	75.9
V2 (+A4 algebra)	67.0	2,598	37.2
V3.1 (+A3 repair)	70.7	1,413	34.0
V5 (30 tokens)	79.7	6	30.6
V5 (512 tokens)	99.0	6	211.1

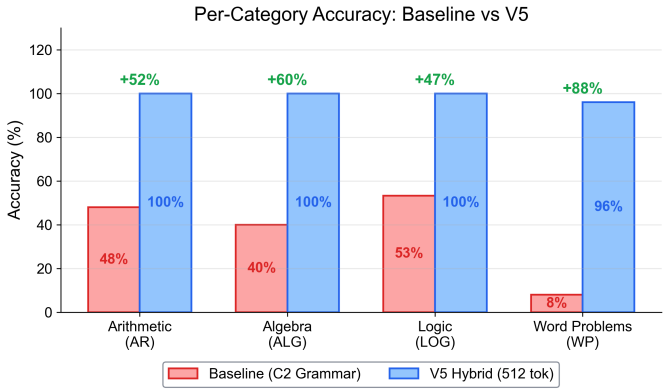

Fig. 6. Per-category accuracy: baseline (C2) vs. V5 (512 tokens). Green annotations show the percentage-point gain per category. AR, ALG, and LOG reach 100% via symbolic solvers; WP improves from 8% to 96% via chain-of-thought prompting.

VIII. ANALYSIS

A. Why Symbolic Routing Works

The accuracy gains from symbolic routing are not incremental: each solver produces an *immediate jump* in its category. Adding A5 moves AR from 48% to 100%; adding A4 moves ALG from 40% to 96–100%; adding A6 moves LOG from 64% to 100%. These reflect the replacement of probabilistic approximation with deterministic computation.

This suggests a design principle: *when the structure of a problem is known, symbolic solvers dominate neural inference in both accuracy and efficiency.* The SLM's value lies in handling queries whose structure is not known in advance. Fig. 11 visualizes the category-level accuracy across all system versions and configurations tested.

B. Error Analysis: The Single V5 Failure

The sole error in V5 (Phi-4-mini, 512 tokens) occurs on prompt WP_000112 from GSM8K:

> *"At Allan's house, there is twice as much corn as cannolis. He has a total of 40 cannolis. Allan bought 60 more cannolis at the store and 40 fewer corns than the number of cannolis. Find the combined total of the number of corns and cannolis Allan has in the house?"*

Expected answer: **200**. Both models consistently produce **240**. The chain-of-thought output reveals the error:

TABLE V
PER-CATEGORY ACCURACY (%): LEARNED ROUTING VS. DEVELOPMENT ABLATION

System	AR	ALG	WP	LOG	All
Learned routing (this work)					
L* routing → V5	**100**	**100**	12	**100**	**78.0**
RPNI routing → V5	96	96	13	92	74.3
Tool-calling agent	95	76	5	59	58.7
V1–V5 ablation (hand-coded routing)					
SLM-only baseline	48	40	8	53	37.3
V5 (30 tokens)	100	100	19	100	79.7
V5 (512 tokens)	100	100	96	100	99.0

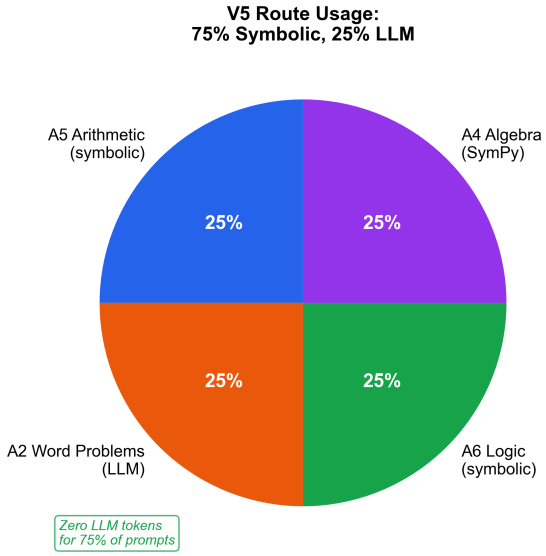

Fig. 7. Route distribution in V5: 75% of queries are dispatched to deterministic symbolic solvers (A5, A4, A6), with only 25% requiring SLM inference (A2 for WP).

> *"Allan bought 60 more cannolis, so now he has* $40+60 = 100$ *cannolis. He also bought 40 fewer corns than the number of cannolis he bought, which means he bought* $100 - 40 = 60$ *corns."*

The model interprets "40 fewer corns than the number of cannolis" as referring to the *updated* cannoli count (100) rather than the original count (40), yielding $100 - 40 = 60$ new corns instead of the intended $40 - 40 = 0$. This is a linguistic ambiguity in the problem statement: the referent of "the number of cannolis" is genuinely unclear. Both Phi-4-mini and Qwen2.5-Math produce the same interpretation across all trials, suggesting this is a systematic parsing bias rather than a stochastic error.

C. Model Comparison

At 512 tokens, both models achieve 99.0% overall and 96.0% WP accuracy. However, at 300 tokens, Phi-4-mini achieves 92.0% WP accuracy versus Qwen's 64.0%, suggesting Phi-4-mini is more token-efficient for word-problem reasoning despite Qwen's mathematics-specific pretraining.

Both models achieve identical performance on symbolic categories (100%) because these queries never reach the model.

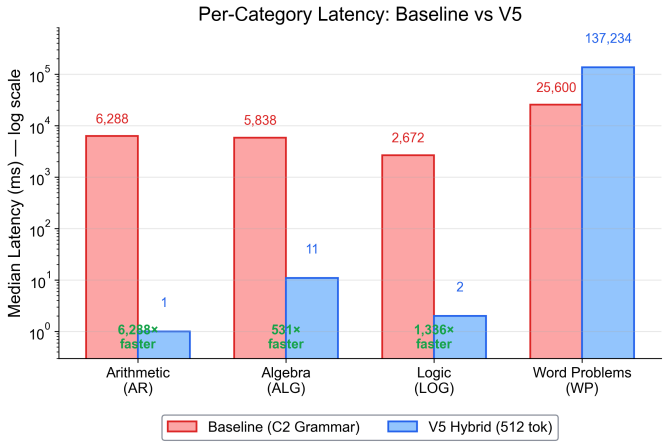

Fig. 8. Per-category median latency on a log scale: baseline vs. V5. Symbolic solvers achieve 530–6,300× speedups on structured categories. WP latency increases due to the higher token budget.

TABLE VI
MEDIAN LATENCY BY CATEGORY

System	AR (ms)	ALG (ms)	LOG (ms)	WP (s)
Baseline (best)	6,288	5,838	2,672	25.6
V5 (512tok)	1	11	2	137.2

IX. DISCUSSION

A. Generalizability

Our benchmark covers four well-defined categories drawn from established datasets. Real-world queries may span additional categories (e.g., commonsense reasoning, spatial reasoning) that are neither fully symbolic nor standard word problems. However, because the routing patterns are *learned* rather than hard-coded, extending to new categories requires only additional training examples and a new DFA, not manual regex engineering.

B. Classifier Robustness

The L*-learned classifier achieves 100% routing accuracy on held-out T2 prompts, but this reflects the well-separated nature of the benchmark categories rather than adversarial robustness. The token-class abstraction (Section IV-A) provides some robustness to paraphrasing, since surface words are mapped to abstract types, but adversarially crafted queries could still cause misclassification. A fallback mechanism that detects solver failures and re-routes would further improve robustness.

C. Active vs. Passive Learning

The comparison between L* (100% classification, 78.0% task accuracy) and RPNI (86% classification, 74.3% task accuracy) demonstrates the value of the SLM membership oracle. RPNI's lower performance stems from word-problem misclassification: WP prompts have diverse linguistic structure that is difficult to capture from 10 positive examples alone. The SLM oracle allows L* to generalize beyond the training distribution by answering membership queries on novel

TABLE VII
WORD-PROBLEM ACCURACY VS. TOKEN BUDGET

Model	Tok.	CoT	WP (%)	Lat. (s)	E (mWh/p)
Phi-4-mini	30	No	18.7	26.0	30.6
Phi-4-mini	300	Yes	92.0	128.6	147.2
Phi-4-mini	512	Yes	96.0	135.2	211.1
Qwen-1.5B	30	No	18.7	26.0	30.6
Qwen-1.5B	300	Yes	64.0	100.1	113.1
Qwen-1.5B	512	Yes	96.0	128.2	147.6

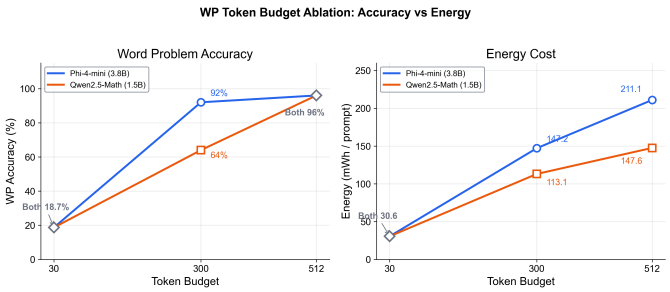

Fig. 9. Word-problem token budget ablation for two SLMs: accuracy (left) and energy cost (right) as a function of token budget. Both models converge to 96% at 512 tokens, but Phi-4-mini is more token-efficient at 300 tokens.

token sequences. This suggests that even weak oracles (a 3.8B parameter model) can meaningfully improve grammatical inference when the target language has high variance.

D. Dataset Size

Our benchmark of 100 prompts (25 per category) is small by modern standards. The 95% confidence interval on 99.0% accuracy is [97.1%, 99.8%], which is reasonably tight, but per-category confidence intervals are wider (e.g., 96% WP accuracy on 25 prompts has a 95% confidence interval of [80.5%, 99.3%]). Evaluation on larger subsets of the source benchmarks would strengthen the claims.

E. Energy–Accuracy Tradeoff

The best-accuracy system (V5, 512tok) uses 2.8× more energy than the baseline. The energy savings from symbolic routing on 75% of queries are real but are outweighed by the 17× increase in WP generation cost. For battery-powered deployments, adaptive token budgets that start low and escalate only when the model's output indicates insufficient reasoning could reduce average energy while maintaining accuracy.

F. Scalability of Symbolic Solvers

Our solvers handle single-variable linear equations, basic arithmetic, and propositional logic. Extending to nonlinear equations, multi-variable systems, or first-order logic would require more powerful solvers, but the routing architecture would remain unchanged.

G. Practical WP Latency

Word problems take approximately 137 s per prompt at 512 tokens on the Pi 4B's CPU. This is acceptable for batch processing or non-interactive applications but prohibitive for real-time use. Faster hardware (e.g., with neural accelerators) or more aggressive quantization could reduce this.

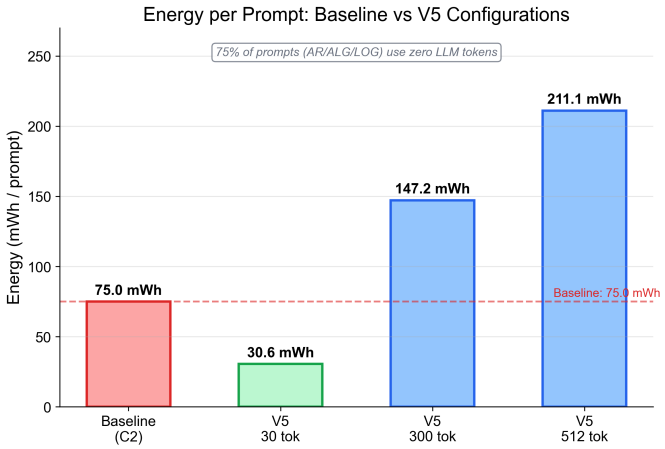

Fig. 10. Energy per prompt (mWh) across system configurations. Higher token budgets (300, 512) dramatically increase energy cost. V5 Default (30 tokens) achieves the lowest energy among hybrid systems.

TABLE VIII
ERROR DISTRIBUTION (300 TRIALS EACH). E0 = CORRECT

System	E0	E1	E2	E3	E5	E7	E8
Baseline	112	36	26	45	21	9	51
V1 (+A5)	159	0	27	45	24	0	45
V2 (+A4)	201	0	27	3	24	0	45
V3.1 (+A3)	212	0	27	0	18	30	13
V5 (512)	297	0	0	0	3	0	0

X. CONCLUSION

We presented a neurosymbolic routing architecture for edge devices that learns query classification patterns via the L* grammatical inference algorithm, using the SLM as a membership oracle. Evaluated on a Raspberry Pi 4B with no GPU, L*-learned routing achieves 78.0% end-to-end task accuracy on 100 held-out prompts, outperforming a tool-calling agent baseline (58.7%) that has access to the same symbolic solvers. The learned router is 2.4× faster and 2.3× more energy-efficient, because structured queries (arithmetic, algebra, logic) are dispatched directly to symbolic solvers in 1–11 ms without invoking the SLM.

A comparison between L* (active, oracle-guided) and RPNI (passive, example-only) demonstrates that even a weak SLM oracle significantly improves routing accuracy (100% vs. 86%), particularly for linguistically diverse categories like word problems. An incremental ablation across five system versions further shows that each symbolic solver contributes a clear accuracy gain in its target category.

The system's remaining weakness is word problems, which are challenging for all approaches tested (12% for learned routing at 30 tokens, 5% for the tool-calling agent). With increased token budgets (512 tokens) and hand-coded routing, WP accuracy reaches 96%, but at 6.9× higher energy. Future work will explore adaptive token allocation, larger evaluation

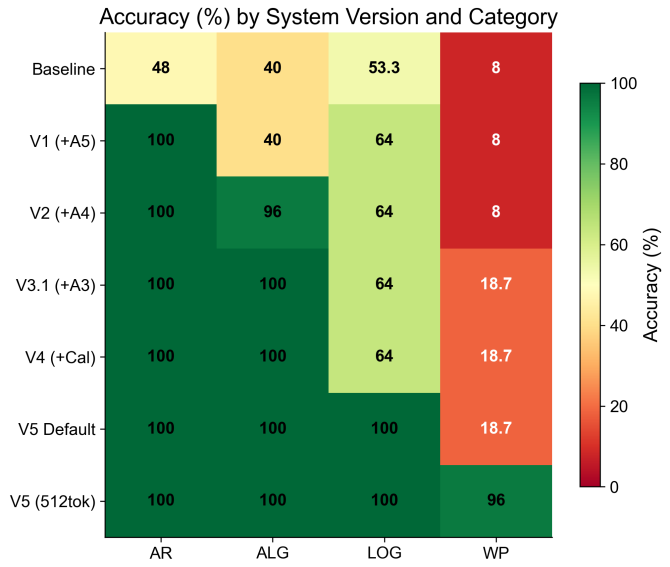

Fig. 11. Accuracy by category across all system versions and configurations. Each column is a category; each row is a system version. Symbolic solvers (A5, A4, A6) produce immediate 100% accuracy in their target categories upon introduction.

benchmarks, and extending grammatical inference to richer language classes. More broadly, this work demonstrates that trustworthy reasoning on edge devices does not require larger models or cloud connectivity; it requires learning to route each query to the right solver.

References

[1] Z. Xu, Y. Xu, J. Li *et al.*, "A survey on efficient inference for large language models," *arXiv preprint arXiv:2404.14294*, 2024.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì *et al.*, "Toolformer: Language models can teach themselves to use tools," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[3] D. Saxton, E. Grefenstette, F. Hill, and P. Kohli, "Analysing mathematical reasoning abilities of neural models," in *International Conference on Learning Representations*, 2019.

[4] K. Cobbe, V. Kosaraju, M. Bavarian *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[5] P. Clark, O. Tafjord, and K. Richardson, "Transformers as soft reasoners over language," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2020.

[6] D. Angluin, "Learning regular sets from queries and counterexamples," *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987.

[7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2023.

[9] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," *arXiv preprint arXiv:2306.11695*, 2024.

[10] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[11] G. Gerganov, "llama.cpp: LLM inference in pure C/C++," https://github.com/ggerganov/llama.cpp, 2023, accessed: 2026-03-01.

[12] A. d. Garcez and L. C. Lamb, "Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning," *Journal of Applied Logics*, vol. 6, no. 4, pp. 611–632, 2019.

[13] H. Kautz, "The third AI summer: AAAI robert s. engelmore memorial lecture," *AI Magazine*, vol. 43, no. 1, pp. 93–104, 2022.

[14] L. Gao, A. Madaan, S. Zhou *et al.*, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[15] E. M. Gold, "Language identification in the limit," *Information and Control*, vol. 10, no. 5, pp. 447–474, 1967.

[16] C. de la Higuera, *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press, 2010.

[17] J. Oncina and P. García, "Inferring regular languages in polynomial updated time," in *Pattern Recognition and Image Analysis*. World Scientific, 1992, pp. 49–61.

[18] P. Fiterău-Broştean, R. Janssen, and F. Vaandrager, "Analysis of security protocols using finite-state machines," *Software Tools for Technology Transfer*, vol. 22, pp. 599–620, 2020.

[19] D. Peled, M. Y. Vardi, and M. Yannakakis, "Black box checking," in *Journal of Automata, Languages and Combinatorics*, vol. 7, no. 2, 2001, pp. 225–246.

[20] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[21] I. Ong, A. Almahairi, V. Wu *et al.*, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[22] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[23] X. Wang, J. Wei, D. Schuurmans *et al.*, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2023.